Design and Development of a Real-Time WDM Optical Network Monitoring and Performance Analysis System

Complete Working Explanation and Theory Tutorial

A step-by-step guide to every module, every equation and every operation in the application

Covers: configuration, fibre models, WDM system, transmitter-receiver, noise models, amplifier, performance, traffic-management, fault-recovery, simulation, visualizations, application, run-examples and the complete test suite

Prepared by: A S Abinesh Raj

Subject area: Electronics and Communication Engineering (Fibre Optical Communication)

Contents

This tutorial is organised in the same order in which a light pulse travels through the simulated system. Each chapter first states the purpose, then the theory, then explains exactly how the code performs each operation, and finally suggests something to try in the dashboard.

1. **Chapter 1** – What the application is and how it is organised
2. **Chapter 2** – Fundamental theory you need first (decibels, dBm, light as a carrier, intensity modulation)
3. **Chapter 3** – Configuration: how every setting is stored (configuration)
4. **Chapter 4** – The optical fibre: geometry, modes, attenuation, dispersion (fibre models)
5. **Chapter 5** – The WDM channel grid, multiplexer and demultiplexer (WDM system)
6. **Chapter 6** – The transmitter and the receiver (transmitter-receiver)
7. **Chapter 7** – The optical amplifier, EDFA (amplifier)
8. **Chapter 8** – Noise models (noise models)
9. **Chapter 9** – The link budget and per-channel performance: SNR, Q-factor, BER, eye diagram (performance)
10. **Chapter 10** – Dispersion and its compensation
11. **Chapter 11** – Traffic management and load balancing (traffic management)
12. **Chapter 12** – Fault detection and protection switching (fault recovery)
13. **Chapter 13** – The orchestrator (simulation)
14. **Chapter 14** – The dashboard, tab by tab (application)
15. **Chapter 15** – The plotting layer (visualizations)
16. **Chapter 16** – Examples and the automated tests
17. **Chapter 17** – A complete worked example with the default settings
18. **Chapter 18** – Limitations, parameter provenance and practice exercises
19. **Appendix** – Symbol table, key formulas and glossary

Chapter 1: What the Application Is and How It Is Organised

1.1 Purpose of the application

The application is a physics-based simulator of a Wavelength Division Multiplexing (WDM) optical fibre communication link. WDM is the technique of sending many independent data streams through one optical fibre at the same time, each stream carried by light of a slightly different colour (wavelength). The simulator lets a student change the fibre type, the length, the number of channels, the amplifier, the noise sources, the dispersion compensation, the network traffic and even inject a channel failure, and then immediately see the effect on received power, signal-to-noise ratio, Q-factor, bit error rate (BER), eye diagram, traffic throughput and recovery time.

The simulator is deliberately a simplified model. It uses textbook equations, not measurements from real hardware. The file config.py states this clearly: every number is either **Sourced** (a physical constant or textbook value), **Calculated** (derived from an equation) or **Assumed** (a representative value chosen by the author). Nothing is measured from real equipment.

1.2 How to run it

The application is a Streamlit web dashboard. The required libraries are listed in requirements.txt: streamlit, numpy, scipy, pandas, plotly, matplotlib, pytest and tabulate. The steps are:

1. Install the libraries with: pip install -r requirements.txt
2. Start the dashboard with: streamlit run app.py
3. A browser tab opens. The left sidebar has the Run Simulation button, an Auto-update checkbox, a Reset to Defaults button, a channel selector and a random-seed control. The main area has twelve tabs.
4. Run the automated tests with: pytest
5. Generate the example configurations with: python run_examples.py

1.3 The files and what each one does

File	Role in the system
config.py	Defines every setting as a Python dataclass, the physical constants, and the table of parameter provenance.
fiber_models.py	Fibre presets, numerical aperture, V-number, number of modes, attenuation, chromatic / modal / polarisation-mode dispersion, pulse broadening budget, validation.
wdm_system.py	CWDM and DWDM wavelength grids, frequency-wavelength conversion, MUX / DEMUX filter model, crosstalk matrix, validation.

File	Role in the system
transmitter_receiver.py	Bit generation, NRZ / RZ on-off keying, extinction ratio, transmitter rise time, receiver filters, decision threshold, BER formulas, eye traces.
noise_models.py	Thermal, shot, laser RIN, signal-ASE beat, ASE-ASE beat, crosstalk and background noise variances and time-domain noise waveforms.
amplifier.py	EDFA noise factor, ASE power spectral density, gain saturation, quantum limit.
performance.py	Link budget (power through every stage), per-channel evaluation (SNR, Q, BER, eye), Monte-Carlo simulation, sweeps and comparisons.
traffic_management.py	Allocation of offered traffic to channels, priority scheduling, adaptive load balancing, packet error rate, latency.
fault_recovery.py	Failure detection time, protection switching, traffic before / during / after a fault, timeline.
simulation.py	Runs the whole chain in the correct order and collects everything into one results object.
visualizations.py	All Plotly figure builders used by the dashboard.
app.py	The Streamlit user interface: sidebar, twelve tabs, state handling, caching, export.
run_examples.py	Builds ready-made example configurations and sample results.
test_*.py (six files)	Automated tests that check the physics, the traffic logic, the fault logic and the dashboard.

1.4 The signal path (the pipeline)

Everything in the simulator follows the path of the light. The comment at the top of performance.py states it exactly:

> P_launch -> MUX -> [booster amp] -> fibre (+ in-line amps) -> [DCF] -> [pre-amp] -> DEMUX -> photodiode -> receiver filter -> decision

Around this optical path, the simulator adds a network layer (traffic management) and a supervisory layer (fault detection and recovery). The overview tab of the dashboard draws this chain as a block diagram with coloured status for each stage.

1. **Configuration** – the user settings are collected into one SystemConfig object.
2. **Data generation** – a pseudo-random bit sequence is generated for each channel.
3. **Transmitter** – bits become an optical power waveform (light on for 1, nearly off for 0).
4. **Multiplexer** – all wavelengths are combined into one fibre; the MUX adds insertion loss.
5. **Fibre** – the light is attenuated (loses power) and dispersed (pulses spread in time).

6. **Amplifier (optional)** – the EDFA restores power but adds ASE noise.
7. **Demultiplexer** – wavelengths are separated again; some light of neighbours leaks through (crosstalk).
8. **Receiver** – a photodiode converts light to current, a filter shapes it, noise is added, and a decision circuit recovers the bits.
9. **Performance analysis** – SNR, Q, BER and eye opening are computed.
10. **Traffic management** – the network decides how much traffic each channel carries.
11. **Fault recovery** – failures are detected and traffic is re-routed.

Chapter 2: Fundamental Theory You Need First

2.1 Why optical fibre and why WDM

An optical fibre is a thin strand of glass (or plastic) that guides light by total internal reflection. Light carries information when its intensity is switched on and off at a very high rate. A single wavelength can carry perhaps 10 Gb/s with simple equipment. Instead of laying many fibres, WDM puts many wavelengths in the same fibre. If eight channels each carry 10 Gb/s, the fibre carries 80 Gb/s. This is exactly the default configuration of the application: 8 channels × 10 Gb/s = 80 Gb/s nominal capacity.

2.2 The decibel (dB) and dBm

Optical powers change by large factors, so engineers use a logarithmic scale. A power ratio expressed in decibels is:

$$\text{Gain or loss (dB)} = 10 \log_{10} \left(P_{out} / P_{in} \right)$$

A loss of 3 dB means the power is halved (the ratio is 0.501). A loss of 10 dB means one tenth of the power remains. The great advantage of decibels is that losses of successive stages simply add. Absolute power is expressed in dBm, which is decibels relative to 1 milliwatt:

$$P\ (\text{dBm}) = 10 \log_{10} \left(P / 1\ \text{mW} \right)$$

So 0 dBm is 1 mW, +3 dBm is about 2 mW, and -20 dBm is 10 microwatts. In performance.py the two conversion functions are:

```
def dbm_to_w(dbm):  return 1e-3 * 10.0 ** (dbm / 10.0)
def w_to_dbm(w):    return 10.0 * np.log10(max(w, 1e-30) / 1e-3)
```

The max(w, 1e-30) protects the logarithm from taking the log of zero when a channel is dead.

2.3 Wavelength, frequency and the speed of light

Light of wavelength λ has frequency ν given by:

$$c = \lambda \nu \qquad \text{so} \qquad \nu = c / \lambda$$

where c = 299 792 458 m/s. A wavelength of 1550 nm corresponds to about 193.4 THz. WDM grids are defined by international standards (ITU-T) either in frequency spacing (DWDM) or in wavelength spacing (CWDM). The functions wavelength_to_frequency_hz and frequency_to_wavelength_nm in wdm_system.py perform this conversion.

2.4 Intensity modulation and direct detection (IM/DD)

The simulator models the simplest and most common system type, called intensity modulation with direct detection, also written IM/DD. At the transmitter a laser is switched between a high power level P1 (a logical 1) and a low power level P0 (a logical 0). This is called on-off keying (OOK). At the receiver a photodiode produces a current proportional to the optical power:

$$\mathbf{I = R \times P}$$

where R is the responsivity in amperes per watt (default 0.9 A/W, a typical value for an InGaAs PIN photodiode at 1550 nm). A decision circuit compares the sampled current with a threshold. Above the threshold it decides 1; below it decides 0. The phase of the light is not used; this is why the laser linewidth does not directly cause errors in this model.

2.5 What limits a fibre link

Three physical effects limit how far and how fast data can be sent, and the whole application is built around them:

- **Attenuation** – the light becomes weaker with distance, so less power reaches the receiver and the signal drowns in noise.
- **Dispersion** – different colours or different paths travel at different speeds, so pulses spread and overlap (inter-symbol interference, ISI).
- **Noise** – random fluctuations (thermal, shot, laser, amplifier, crosstalk) add uncertainty to the received level, causing wrong decisions (bit errors).

Chapter 3: Configuration (config.py)

3.1 Purpose

Before anything can be simulated, all settings must be stored in an organised way. config.py does this with Python dataclasses. A dataclass is a simple class that holds named fields with default values. The advantage is that the same object can be created with defaults, modified, copied, converted to JSON text and rebuilt from JSON. This is how the dashboard caches results and how settings can be exported.

3.2 Physical constants (sourced)

Constant	Symbol	Value in the code
Speed of light	c	299 792 458 m/s
Planck constant	h	6.62607015 × 10^-34 J s
Elementary charge	q	1.602176634 × 10^-19 C
Boltzmann constant	k	1.380649 × 10^-23 J/K
Maximum channels in the UI	MAX_CH	16
Channel healthy limit	BER_OK	1 × 10^-9
Channel failed limit	BER_FAIL	1 × 10^-3

The two BER limits define the status colours: a channel with BER at or below 1e-9 and received power above the receiver sensitivity is OK; BER above 1e-9 or power below sensitivity is DEGRADED; BER above 1e-3 is FAILED.

3.3 The configuration sections

The full configuration SystemConfig is made of eight sections plus the selected channel number:

Section class	What it controls	Important defaults
FiberConfig	Material, mode structure, index profile, length, core / cladding size, refractive indices, operating wavelength, optional attenuation / dispersion override, PMD, modal bandwidth, connectors, splices, margin	Silica, single-mode, step-index, 20 km, core 8.2 µm, cladding 125 µm, n_core 1.4682, n_clad 1.4633, 2 connectors at 0.5 dB, splice 0.05 dB every 2 km
WDMConfig	DWDM or CWDM, number of channels, grid, custom wavelengths, per-channel bit rate and power, MUX / DEMUX loss, filter bandwidth, order and isolation	DWDM, 8 channels, 100 GHz spacing around 1550.12 nm, 10 Gb/s and 0 dBm per channel, MUX and DEMUX 3 dB each, demux bandwidth 0.5 of spacing, isolation 40 dB

Section class	What it controls	Important defaults
TxRxConfig	Number of bits, Monte-Carlo bits, samples per bit, seed, modulation, extinction ratio, rise time, RIN, linewidth, spectral width, responsivity, receiver bandwidth and filter, sensitivity, dark current, temperature, load resistance, receiver noise figure, jitter, threshold type	NRZ-OOK, 256 bits shown, 20 000 bits in Monte-Carlo, 16 samples per bit, extinction ratio 10 dB, rise 0.25 of a bit, RIN -150 dB/Hz, responsivity 0.9 A/W, receiver bandwidth 0.7 × bit rate, Bessel filter, sensitivity -24 dBm, 300 K, 500 ohm, NF 3 dB
NoiseConfig	On / off switches for each noise source	Thermal, shot, RIN, ASE and crosstalk ON; background and jitter OFF
AmpConfig	Amplifier on / off, placement, gain, noise figure, number of in-line amplifiers, saturation and output power limit	OFF, pre-amplifier, 15 dB, NF 5 dB, saturation limit 17 dBm
DispConfig	Dispersion on / off, compensation on / off, method (DSP or DCF), compensated fraction, DCF loss	Dispersion ON, compensation OFF, DSP, 0.9 fraction, DCF loss 5 dB
TrafficConfig	Demand per channel, priority, compatibility group, congestion threshold, balancing and scheduling switches, packet size, re-route latency	Demands 9.5, 6, 3, 8.8, 2, 5, 4, 7 Gb/s; threshold 0.8; 1500-byte packets; 50 µs re-route overhead
FaultConfig	Failed channels, protection on / off, backup mode and channel, failure time, poll period, debounce count, switch time, timeline end	No failures, protection ON, failure at 100 ms, poll 10 ms, debounce 3, switch 50 ms, end 400 ms

3.4 JSON conversion

SystemConfig provides to_dict, to_json, from_dict and from_json. to_json uses sort_keys=True so that two identical configurations always give the identical text string. The dashboard uses this string as a cache key: if the text has not changed, the previous simulation result is reused and nothing is recomputed. from_dict is tolerant: it ignores unknown keys and fills missing keys with defaults, so older saved files still load.

3.5 Parameter provenance table

At the end of config.py is the list PARAMETER_PROVENANCE. It records, for every model, whether it is Sourced, Calculated or Assumed. For example, the Rayleigh scattering form of silica attenuation is sourced from Agrawal but the constants A and B are representative; the EDFA noise theory is sourced; the 50 ms protection switching time is assumed. This honesty table is shown in the Summary tab so that nobody mistakes simulated numbers for measurements.

Try it: In a Python shell run: from config import SystemConfig; c = SystemConfig(); print(c.to_json()[:200]). Then change c.fiber.length_km = 50 and compare the JSON text.

Chapter 4: The Optical Fibre (fiber_models.py)

4.1 Purpose

This module answers the question: what does a length L of this particular fibre do to light of wavelength λ? It computes the guiding geometry, how many modes propagate, how much power is lost, and how much a pulse spreads in time.

4.2 Fibre structure and light guiding

A fibre has a central core of refractive index n_core surrounded by a cladding of slightly lower index n_clad. Light is guided in the core by total internal reflection because n_core is greater than n_clad. The code checks this in validate_fiber: if n_core is not greater than n_clad it reports an error because no guiding is possible.

The simulator supports three materials and two structures:

- **Silica** – glass fibre, the standard for telecommunications; usable 600 to 1700 nm in this model.
- **POF** – plastic optical fibre (PMMA), a very large core (980 µm) used for short links; usable 450 to 800 nm because plastic is opaque beyond about 850 nm.
- **PCS** – plastic-clad silica, a glass core with polymer cladding, 200/230 µm; usable 600 to 1650 nm.
- **Single-mode** – the core is so small that only one light path (mode) propagates. No modal dispersion.
- **Multimode** – a large core carries many modes; different modes arrive at different times (modal dispersion).
- **Step-index** – the refractive index changes abruptly between core and cladding.
- **Graded-index** – the core index decreases smoothly from the centre, which equalises mode travel times and greatly reduces modal dispersion.

The dictionary FIBER_PRESETS holds a representative parameter set for every combination of (material, mode, profile): core and cladding diameters, indices, attenuation coefficients A and B, the OH peak amplitude, PMD, effective modal bandwidth, the zero-dispersion wavelength lam0 and the dispersion slope s0. Each preset has a status of ok, warn or invalid. For example single-mode POF is marked invalid because it is not a commercial product; the dashboard shows a warning rather than silently accepting it. The function defaults_for returns the values placed into the dashboard when the user changes the material, mode or profile selection (and chooses a typical operating wavelength: 650 nm for POF, 850 nm for PCS and multimode, 1550 nm for single-mode silica).

4.3 Numerical aperture, V-number, cutoff and mode count

Numerical aperture (NA)

The numerical aperture measures how wide a cone of light the fibre can accept:

$$\mathbf{NA = \sqrt{n_{core}^2 - n_{clad}^2}}$$

For the default fibre: $\sqrt{1.4682^2 - 1.4633^2} = 0.1199$. The function numerical_aperture uses max(..., 0) so that an invalid index pair does not produce an imaginary number.

Normalised frequency (V-number)

$$\mathbf{V = 2\pi\, a\, NA / \lambda}$$

where a is the core radius. In the code the core diameter in micrometres is halved and the wavelength in nanometres is divided by 1000 so that both are in micrometres. V tells how many modes the core supports. For the default fibre at the average channel wavelength, V = 1.99.

Single-mode condition and cutoff wavelength

A step-index fibre supports only one mode when V is less than or equal to 2.405, the first zero of the Bessel function J0. This constant is SM_V_CUTOFF in the code. The cutoff wavelength is the wavelength below which the fibre becomes multimode:

$$\mathbf{\lambda_c = 2\pi\, a\, NA / 2.405}$$

For the default fibre $\lambda_c = 1284$ nm. Since the operating wavelength is about 1550 nm, which is greater than 1284 nm, V = 1.99 is less than 2.405, so the fibre is single-mode. This is displayed in the Fibre tab.

Number of guided modes

For large V the approximate number of modes is $V^2/2$ for step-index fibre and $V^2/4$ for graded-index fibre. For V at or below 2.405 the code returns exactly 1. The function num_modes implements this estimate.

Relative index difference

$$\mathbf{\Delta = (n_{core} - n_{clad}) / n_{core}}$$

For the default fibre $\Delta = 0.00334$. This small number is central to modal dispersion.

All of this is collected by fiber_geometry, which returns na, v, modes, cutoff_nm, delta and a Boolean single_mode.

4.4 Attenuation

Theory

Light power decreases exponentially with distance. In decibels the loss is linear with distance:

$$\text{Loss (dB)} = \alpha\ \text{(dB/km)} \times L\ \text{(km)}$$

The attenuation coefficient α depends strongly on wavelength. For silica three mechanisms matter: Rayleigh scattering (tiny density fluctuations scatter light, proportional to $1/\lambda^4$), infrared absorption by the glass lattice at long wavelengths (rises steeply beyond about 1600 nm) and OH ion absorption (a peak near 1383 nm caused by water in the glass). The code models this as:

$$\alpha(\lambda) = A / \lambda^4 + B + \text{IR tail} + \text{OH peak}$$

```
ir = 7.81e11 * np.exp(-48.48 / lam)                 # infrared absorption tail
oh = p["oh"] * np.exp(-0.5 * ((lam - 1.383) / 0.025) ** 2)  # OH peak (Gaussian)
return p["A"] / lam ** 4 + p["B"] + ir + oh          # lam in micrometres
```

For the default single-mode silica preset (A = 0.75, B = 0.04, OH amplitude 0.12) the result is about 0.19 dB/km near 1550 nm, which is the well-known low-loss window of silica fibre. The operating windows used in practice are about 850 nm (first window), 1310 nm (second) and 1550 nm (third); a test checks the model against these textbook values.

For POF the code interpolates a table of attenuation values (450 nm: 200 dB/km, 570 nm: 90 dB/km, the green minimum, 650 nm: 150 dB/km, 800 nm: 3000 dB/km) using logarithmic interpolation, and sets 20 000 dB/km beyond 850 nm so that the fibre is effectively opaque. For PCS a representative constant in the 4 to 6 dB/km class is built from the B term.

Manual override

If the user ticks Manual attenuation, attenuation_db_km does not replace the curve but scales the whole curve so that the value at the operating wavelength equals the typed number. This keeps the wavelength dependence between channels realistic.

4.5 Chromatic dispersion

Theory

The refractive index of glass depends on wavelength, so different colours travel at different speeds. A real laser pulse contains a small spread of wavelengths, so its parts arrive at slightly different times and the pulse widens. The dispersion parameter D is measured in picoseconds of delay per nanometre of spectral width per kilometre of fibre (ps/nm/km). For standard single-mode fibre the ITU-T G.652 form is:

$$D(\lambda) = (S0 / 4) \times (\lambda - \lambda0^4 / \lambda^3)$$

where $\lambda 0 = 1313$ nm is the zero-dispersion wavelength and $S0 = 0.092$ ps/nm²/km is the slope at that point. At 1550 nm this gives $D \approx 17$ ps/nm/km; in the default run $D = 17.14$. D is zero near 1313 nm and positive at 1550 nm. For POF the model uses a constant -300 ps/nm/km.

4.6 Modal dispersion

In a multimode fibre, rays that travel at steeper angles take a longer path than the axial ray. In a step-index fibre the delay difference between the slowest and fastest ray over length L is approximately $L\, n1\, \Delta / c$. The RMS broadening used in the code is:

$$\sigma_modal\ (step\text{-}index) = L\, n1\, \Delta / (2\sqrt{3}\, c)$$

For graded-index fibre the ideal parabolic profile greatly reduces this:

$$\sigma_modal\ (graded, ideal) = L\, n1\, \Delta^2 / (20\sqrt{3}\, c)$$

Real graded-index fibre is limited by its effective modal bandwidth (EMB) in MHz·km. The code converts this to an RMS width using $s = 0.1874 / (EMB / L)$ and takes the larger of the ideal result and the EMB result. If the fibre is physically single-mode ($V \leq 2.405$) modal dispersion is exactly zero. Notice the dramatic difference: a 50 µm step-index multimode fibre has a Δ about 100 times larger than a graded one has Δ^2, so step-index fibre is only usable for very short distances.

4.7 Polarisation-mode dispersion (PMD)

Real fibres are slightly non-circular and the two polarisations travel at slightly different speeds. The mean delay grows with the square root of length because the imperfections are random:

$$PMD\ (ps) = PMD\ coefficient\ (ps/\sqrt{km}) \times \sqrt{L}$$

With the default coefficient 0.1 ps/√km and L = 20 km, PMD = 0.447 ps, which is negligible compared with the chromatic broadening of 12.8 ps. The function pmd_rms_ps returns zero for multimode fibre because modal dispersion dominates there.

4.8 The dispersion budget: dispersion_budget

This is the most important function of the fibre module. It combines all dispersion terms into the extra RMS pulse width added by the fibre. Follow its operations one by one:

1. **Bit period** $T_b = 1 /$ bit rate. For 10 Gb/s, $T_b = 100$ ps.
2. **Input RMS width of an ideal NRZ bit:** $\sigma_in = T_b / \sqrt{12} = 28.87$ ps (the RMS width of a rectangle).
3. **Spectral width of the source.** The width of the optical spectrum has three contributions added in quadrature: the source chirp width (spectral_width_nm = 0.03 nm), the laser linewidth converted from frequency to wavelength ($d_lw = \lambda^2 \Delta\nu / c$), and the minimum

width forced by the modulation itself ($d_{mod} = \lambda^2 / (4\pi c \sigma_{in})$). The total is $\sigma_\lambda = \sqrt{(chirp^2 + d_{lw}^2 + d_{mod}^2)}$. For the default case $\sigma_\lambda = 0.0372$ nm.

4. **Chromatic broadening:** $\sigma_{chrom} = |D| \times L \times \sigma_\lambda = 17.14 \times 20 \times 0.0372 = 12.75$ ps.
5. **Modal and PMD broadening** from sections 4.6 and 4.7.
6. **Switch for dispersion on / off.** If dispersion is disabled all three terms are set to zero.
7. **Compensation.** If compensation is enabled, the chromatic term is multiplied by (1 - comp_fraction). Modal dispersion and PMD are not compensated.
8. **Quadrature sum:** extra width = $\sqrt{(\sigma_{chrom}^2 + \sigma_{modal}^2 + \sigma_{pmd}^2)}$. Independent random spreads add in power, hence squares are added.
9. **Output pulse width:** $\sigma_{out} = \sqrt{(\sigma_{in}^2 + extra^2)} = 31.56$ ps in the default case.
10. **Dispersion power penalty** (Gaussian-pulse approximation from Agrawal):

$$\textbf{Penalty (dB)} = 5 \log_{10} (1 + 2\pi (B \sigma)^2)$$

where B is the bit rate and σ is the extra width. The default case gives 0.21 dB. The result dictionary also stores the uncompensated values (raw_ps, penalty_raw_db) so that before-and-after comparisons are possible.

4.9 Validation: validate_fiber

Instead of silently running with a physically wrong setup, the function returns a list of (level, message) pairs, with level error, warning or info. It checks: invalid or unusual combinations; cladding not larger than core; n_core not greater than n_clad; channel wavelengths outside the model range; unusual numerical aperture (above 0.6); a fibre labelled single-mode whose V is greater than 2.405 at the shortest wavelength (it then says that the fibre is physically multimode and modal dispersion is applied); a fibre labelled multimode whose V is at or below 2.405; a graded-index fibre with no modal bandwidth (an optimistic ideal profile is assumed); and a non-positive length. Errors stop the simulation; warnings are shown in yellow boxes.

Try it: In the Fibre tab change the structure to Multimode and the profile to Step-index, set length to 5 km and watch the modal dispersion value in the table. Then change the profile to Graded-index and compare.

Chapter 5: The WDM Grid, Multiplexer and Demultiplexer (wdm_system.py)

5.1 Purpose

This module decides which wavelengths the channels use, checks that the grid is physically sensible, and models how the multiplexer and demultiplexer combine and separate the channels, including the unwanted leakage between neighbouring channels (crosstalk).

5.2 CWDM and DWDM

Property	CWDM (Coarse WDM)	DWDM (Dense WDM)
Standard	ITU-T G.694.2	ITU-T G.694.1
Spacing	20 nm (wavelength grid)	12.5, 25, 50, 100 or 200 GHz (frequency grid)
Channels	Up to 18 (1271 to 1611 nm)	Dozens in the C and L bands
Lasers and filters	Cheap, uncooled	Precise, temperature-controlled
Typical use	Metro and access networks	Long-haul and backbone with EDFAs

5.3 How channel wavelengths are calculated: channel_wavelengths

There are three branches in the code:

1. **Custom wavelengths.** If the user enabled them, the first n values typed in the table are sorted ascending and used.
2. **CWDM.** The wavelengths are an arithmetic progression: $\lambda_k = \text{start} + \text{spacing} \times k$, with default start 1471 nm and spacing 20 nm, giving 1471, 1491, 1511 ... nm.
3. **DWDM.** The grid is built in frequency, because the standard defines it in frequency. First the centre frequency $f_0 = c / 1550.12\ \text{nm} = 193.4\ \text{THz}$ is computed. Then the n channel frequencies are $f_k = f_0 + (k - (n - 1)/2) \times \Delta f$, with $\Delta f = 100\ \text{GHz}$, so the grid is symmetric around the centre. Finally the frequencies are converted back to wavelengths and sorted. Channel 1 is the shortest wavelength.

Why does 100 GHz correspond to about 0.8 nm? Differentiating $c = \lambda\nu$ gives $\Delta\lambda = \lambda^2 \Delta\nu / c = (1550 \times 10^{-9})^2 \times 100 \times 10^9 / (3 \times 10^8) = 0.80\ \text{nm}$. The first default channel is 1547.32 nm and the eighth is 1552.93 nm, an overall span of about 5.6 nm. A test (test_dwdm_100ghz_spacing_is_about_0p8_nm) checks exactly this.

5.4 Grid validation: validate_wdm

The function returns issues in the same (level, message) format. Its checks are:

- Number of channels between 1 and 16; fewer custom wavelengths than channels is an error; zero or negative wavelengths are an error; duplicate wavelengths are an error.
- Bit rate of every channel must be positive.
- **Spacing versus bit rate.** If the minimum spacing in GHz is below the bit rate, the channel spectra overlap (error). A modulated signal occupies roughly its bit rate in bandwidth. If the spacing is less than twice the bit rate, a warning predicts strong crosstalk. With 100 GHz spacing and 10 Gb/s this is comfortable (ten times).
- CWDM: at most 18 channels, nominal 20 nm spacing, channels inside 1260 to 1625 nm, start wavelength on the ITU grid 1271 + 20k nm.
- DWDM: spacing should be a standard value; channels should stay in the C+L band 1530 to 1625 nm; with the amplifier on, channels should stay inside the EDFA C-band gain window of 1528 to 1565 nm because the model assumes flat gain.
- Channels must lie within the fibre material model range.
- Demux bandwidth below 1.2 times the bit rate: a warning that filter-induced ISI is not modelled.

5.5 The demultiplexer filter model

A real demultiplexer (for example an arrayed waveguide grating) is a set of narrow optical band-pass filters, one per channel. The simulator models each filter with a generalised Gaussian shape:

$$T(\Delta f) = \exp\left(- \ln 2 \times \left(2 \, |\Delta f| / BW \right)^{(2n)} \right)$$

where BW is the 3 dB bandwidth and n is the filter order. At $\Delta f = BW/2$ the exponent is ln2 and $T = 0.5$ (that is, -3 dB), which confirms the definition of the 3 dB bandwidth. Order 1 gives a Gaussian shape; higher orders give flatter tops and steeper skirts.

Bandwidth of each filter: demux_bw3db_hz

The filter bandwidth is set as a fraction (default 0.5) of the distance to the nearest neighbouring channel: $BW_i = frac \times \min_j |f_i - f_j|$. The code builds the full distance matrix, sets its diagonal to infinity (so a channel is not its own neighbour), and takes the row minimum. For 100 GHz spacing, BW = 50 GHz.

Noise-equivalent optical bandwidth: optical_noise_bandwidth_hz

Optical noise, such as ASE, passes through the demux filter. The effective noise bandwidth is the integral of the filter shape. For the generalised Gaussian this has the closed form:

$$B_o = BW \times \Gamma(1 + 1/2n) / (\ln 2)^{(1/2n)}$$

where Γ is the gamma function (imported from scipy.special). For order 1, B_o = 1.0645 × BW.

The crosstalk matrix: demux_matrix

The matrix T[i, j] states what fraction of the power of channel j appears at the output port of channel i. The wanted path has T[i, i] = 1 (the loss is handled elsewhere as insertion loss). The unwanted terms are the filter skirt evaluated at the frequency difference Δf = f_j - f_i, plus a flat isolation floor of 10^(-isolation_db/10) (default 40 dB gives 0.0001). The code:

```
df = f[None, :] - f[:, None]
T = np.exp(-np.log(2.0) * (2.0 * np.abs(df) / bw) ** (2 * order))
T = T + 10.0 ** (-isolation_db / 10.0) * (1.0 - np.eye(len(f)))
np.fill_diagonal(T, 1.0)
```

The test test_demux_crosstalk_matrix_properties confirms that the diagonal is 1 and that all off-diagonal values are small and symmetric in structure.

5.6 Other helper functions

- **min_spacing_ghz** – smallest frequency difference between any two adjacent channels.
- **nominal_capacity_gbps** – the sum of per-channel bit rates. This is the nominal capacity, NOT the received throughput. The dashboard uses this wording repeatedly to avoid confusion.

Try it: In the WDM tab reduce the DWDM spacing from 100 GHz to 25 GHz and look at the crosstalk level in the Performance tab. Then increase the bit rate to 40 Gb/s and watch the warnings about spacing.

Chapter 6: The Transmitter and the Receiver (transmitter_receiver.py)

6.1 Purpose and sampling method

This module creates the optical waveform of the transmitter and turns the received light into decisions. All waveforms are sampled at fs = bit rate × samples per bit (default 16 samples per bit). The filters are applied in the frequency domain using the Fast Fourier Transform (FFT) with real, zero-phase responses. Zero phase means the sampling instant stays at the centre of each bit. A note in the module says the bit sequence is treated as periodic because of the FFT wrap-around, which is negligible for hundreds of bits.

6.2 Bit generation: generate_bits

A NumPy random generator produces random 0 and 1 values. The seed is tx.seed + channel index, so every channel has a different but reproducible pattern when Reproducible seed is on. The first two bits are forced to 0 and 1 so that both symbols are guaranteed to be present (otherwise the eye diagram and level statistics could fail).

6.3 Transmitter levels and extinction ratio: tx_levels

A real laser can never be completely off for a logical 0. The extinction ratio r is the ratio P1/P0, normally expressed in dB. The default 10 dB means r = 10. Given the average power P_avg, which is what the user sets as launch power, the code solves for P1 and P0. For equiprobable bits and duty factor d (d = 1 for NRZ, d = 0.5 for RZ, where the 1 is a pulse in the first half of the bit):

$$\mathbf{P0 = 2\ P_avg\ /\ (\ d\ r + 2 - d\ )} \qquad \mathbf{P1 = r\ P0}$$

Check for NRZ: average = (P1 + P0)/2 = P0 (r + 1)/2, and 2 P_avg / (r + 1) × (r + 1)/2 = P_avg. Correct. For the default 0 dBm (1 mW) average and r = 10: P0 = 2/11 mW = 0.182 mW and P1 = 1.82 mW.

6.4 NRZ and RZ modulation

NRZ-OOK (non-return-to-zero) holds the high level for the whole bit period. It needs less bandwidth. **RZ-OOK** (return-to-zero) returns to the low level in the second half of each bit, so the pulse is narrower, needs more bandwidth, but is more tolerant of some effects. The function build_tx_waveform repeats each bit sps times, multiplies by a half-bit window for RZ, scales to the two power levels, and then applies the transmitter rise time.

6.5 Finite rise time

The laser driver cannot switch instantly. The model passes the ideal waveform through a Gaussian filter. The 10% to 90% rise time of a Gaussian-filtered step equals 2.563σ, so the code sets σ = rise_frac × sps / 2.563 samples. With the default rise_frac = 0.25 the rise time is a quarter of a bit period (25 ps at 10 Gb/s).

```
def gaussian_filter(x, sigma_samples):
    f = np.fft.rfftfreq(len(x))
    return _apply(x, np.exp(-2.0 * (np.pi * f * sigma_samples) ** 2))
```

This is the Fourier transform of a unit-area Gaussian of RMS width σ. The helper _apply performs the operation: take the real FFT, multiply by the frequency response H, take the inverse FFT.

6.6 The receiver filter

After the photodiode, an electrical low-pass filter limits noise but also rounds the pulses. The default receiver 3 dB bandwidth is 0.7 × bit rate (7 GHz at 10 Gb/s), a standard rule of thumb. Two filter types are provided:

- **Bessel (default).** Implemented as the Gaussian $|H|^2 = 2^{-(f/fc)^2}$. It has almost no overshoot or ringing, which is the property of a Bessel-Thomson filter.
- **Butterworth.** Magnitude $1/\sqrt{(1 + (f/fc)^{(2n)})}$. It is maximally flat but rings more and therefore adds more ISI.

Noise-equivalent bandwidth. Noise power depends on the integral of $|H|^2$, not on the 3 dB bandwidth. The function nebw_factor returns the ratio: for Bessel (Gaussian) the factor is $0.5\sqrt{(\pi / \ln 2)} = 1.0645$; for Butterworth of order n it is x / sin(x) with $x = \pi / 2n$. The electrical noise bandwidth used throughout the noise formulas is Be = B × rx_bw_frac × nebw_factor.

6.7 Decision threshold

The decision circuit compares the sampled current with a threshold I_th. The code offers two choices:

- **Midpoint:** I_th = (I1 + I0) / 2. Best when both levels have equal noise.
- **Optimum:** I_th = (σ0 I1 + σ1 I0) / (σ0 + σ1). This places the threshold so that the two error probabilities are balanced in terms of standard deviations, giving the minimum total error when the noise at 1 is larger than at 0 (which is the usual case, because shot noise, RIN and signal-ASE beating increase with signal power).

6.8 BER and Q-factor

If the noise on each level is Gaussian with standard deviation σ1 and σ0, an error occurs when a 1 falls below the threshold or a 0 rises above it. For equally likely bits:

$$\mathbf{BER = 1/4 \left[erfc\left((I1 - I_th) / (\sigma1 \sqrt{2}) \right) + erfc\left((I_th - I0) / (\sigma0 \sqrt{2}) \right) \right]}$$

At the optimum threshold this simplifies to the well-known expressions with the Q-factor:

$$\mathbf{Q = (I1 - I0) / (\sigma1 + \sigma0)} \qquad \mathbf{BER = 1/2\ erfc(Q / \sqrt{2})}$$

Useful reference values: Q = 6 gives BER ≈ 1 × 10^-9, Q = 7 gives about 1 × 10^-12. The test test_ber_from_q_reference_values verifies these. The functions are ber_from_levels (general threshold), q_factor and ber_from_q. If a standard deviation is zero (no noise) the code returns 0 for a correctly separated pair and 0.5 otherwise, avoiding a divide-by-zero.

6.9 Recovering bits and the eye diagram

sample_indices gives the centre sample of every bit: k × sps + sps/2. **recover_bits** compares those samples with the threshold. **eye_traces** cuts the waveform into overlapping windows two bit periods long, starting every bit period, and overlays them. If the eye is wide open the traces separate cleanly into a high rail and a low rail with a clear opening in the middle; noise and ISI close the eye.

Try it: In the Tx/Rx tab change the receiver filter to Butterworth with order 6 and see the eye opening fall in the Signals tab. Then reduce the extinction ratio to 4 dB and observe the Q-factor drop because I1 - I0 becomes smaller.

Chapter 7: The Optical Amplifier (amplifier.py)

7.1 Purpose

An Erbium-Doped Fibre Amplifier (EDFA) boosts all WDM channels in the 1530 to 1565 nm C-band simultaneously, without converting them to electrical signals. It is the reason long-haul DWDM is practical. However no amplifier is perfect: it adds amplified spontaneous emission (ASE), a broadband optical noise.

7.2 Noise figure and spontaneous emission factor

The noise figure NF is how much the signal-to-noise ratio degrades through the amplifier. It is related to the spontaneous-emission factor n_sp (a measure of how complete the population inversion is) by:

$$n_{sp} = (NF \times G - 1) / (2 (G - 1))$$

where G is the linear gain and NF is the linear noise figure. This is exact in the code for $G > 1$; n_sp is never allowed to fall below 1 because $n_{sp} = 1$ is the physical limit of full inversion. For G = 15 dB (31.6) and NF = 5 dB (3.16), $n_{sp} = (3.16 \times 31.6 - 1) / (2 \times 30.6) = 1.60$.

7.3 Quantum limit

Setting $n_{sp} = 1$ gives the minimum possible noise figure for a given gain: NF_min = 10 log10((2G - 1) / G), which tends to 3 dB for large gain. quantum_limit_nf_db implements it, and collect_issues in simulation.py warns the user if a noise figure below this limit is entered (the value is then clamped to $n_{sp} = 1$).

7.4 ASE power spectral density

$$S_{sp} = n_{sp} (G - 1) h \nu$$

S_{sp} is the ASE power per unit optical bandwidth per polarisation in W/Hz. The code multiplies by Planck constant h and the channel frequency ν. Total ASE power inside an optical bandwidth B_o, counting both polarisations, is $P_{ASE} = 2 S_{sp} B_o$ (function ase_power_w). The function ase_psd_per_pol returns zero when $G \leq 1$.

7.5 Gain saturation: effective_gain_db

An amplifier has a limited total output power. When the input is strong, the gain reduces. The model is a hard limit, simple and understandable: the effective gain is min(requested gain, $P_{sat,dBm} - P_{in,dBm}$), never below 0 dB, where P_{in} is the total input power of ALL channels. For the default case with 8 channels of about -11.3 dBm each, total input is about -2.3 dBm, so the

limit is 17 - (-2.3) = 19.3 dB and the requested 15 dB is allowed. With more channels or a higher launch power, the gain would be cut and the dashboard shows an information message that the amplifier is output-power limited.

7.6 Amplifier placement

Where the EDFA is placed in the link is chosen in the Amplifier tab:

- **Booster** – immediately after the multiplexer; raises the launch power but noise is added early and then attenuated by the fibre, so the ASE is small at the end.
- **In-line** – the fibre is split into n_inline + 1 equal spans with an amplifier between them; each amplifier compensates the loss of the previous span. ASE from each amplifier accumulates.
- **Pre-amplifier** – immediately before the demultiplexer / receiver; it raises the signal above the receiver thermal noise floor; this is the best position for improving receiver sensitivity and is the default placement.

Chapter 8: Noise Models (noise_models.py)

8.1 Purpose

Noise is the reason bits are received wrongly. This module gives the variance (mean square deviation, in A^2) of each noise current. The standard deviation $\sigma = \sqrt{\text{variance}}$ is what appears in the Q-factor. All noise sources are independent, so their variances add. A source that the user switches off contributes exactly zero (a test verifies this). The noise bandwidth Be is the electrical noise-equivalent bandwidth and Bo is the optical noise-equivalent bandwidth.

8.2 The six noise sources

1. Thermal noise

The random motion of electrons in the load resistor and receiver electronics produces a current noise that is independent of the optical signal:

$$\sigma^2_thermal = (4 k T F / R_L) \times Be$$

k is Boltzmann constant, T the temperature in kelvin (default 300 K), R_L the load resistance (500 ohm) and F the receiver noise factor (3 dB = 1.995). Thermal noise is the same for 0 and 1. It is the largest contributor in the default configuration (0.70 μA RMS).

2. Shot noise

Light arrives as discrete photons that generate discrete electrons. The randomness of this arrival is shot noise, and it grows with the total current:

$$\sigma^2_shot = 2 q (I_total + I_dark) \times Be$$

where q is the electron charge and I_dark is the dark current (1 nA). I_total includes the signal, crosstalk and ASE DC components at that logic level. Because it grows with power, shot noise is higher at logic 1 than at logic 0 (default 0.53 μA).

3. Laser relative intensity noise (RIN)

The laser output power fluctuates slightly on its own. This noise is proportional to the square of the signal power:

$$\sigma^2_RIN = RIN \times I_sig^2 \times Be$$

RIN is given in dB/Hz (default -150 dB/Hz, that is 10^-15 per Hz) and converted to a linear value. Since it depends on I^2, it is much higher at logic 1 than at logic 0, and a test (test_shot_noise_grows_with_power_and_rin_with_square) confirms that RIN grows with the square of power.

4. Amplifier ASE beat noise

At the photodiode, which responds to power, the ASE beats against the signal and against itself. This creates two noise terms:

$$\sigma^2_\text{signal-ASE} = 4\,R^2\,P\,S_sp\,Be$$

$$\sigma^2_\text{ASE-ASE} = 4\,R^2\,S_sp^2\,Bo\,Be\,(\,1 - Be / 2Bo\,)$$

Signal-ASE beating is proportional to the signal power so it is larger for 1s. ASE-ASE beating is independent of the signal. Both are zero if the amplifier is off. If background noise is also on, a cross term $2 \times 4\,R^2\,S_a\,S_b\,Bo\,Be\,(1 - Be/2Bo)$ is added to the ASE group (see the code line in level_variances).

5. Inter-channel crosstalk

Neighbouring channels leak through the imperfect demultiplexer. Since the other channels carry random data, their leaked power fluctuates and acts like noise. The model treats it as incoherent:

$$\sigma^2_\text{crosstalk} = R^2\,\Sigma_j\,(\,T_ij\,P_j\,\kappa\,)^2 \qquad \text{with } \kappa = (\,P1 - P0\,) / 2 \text{ per unit average power}$$

where T_ij is the demux matrix element, P_j the received power of the interfering channel j, and κ the modulation depth. The mean leaked power adds a DC offset (p_xt_mean) that also contributes shot noise. In _rx_noise_params only channels that are not failed are included in the sum, which is why a failed channel stops interfering with its neighbours (a test checks this). The crosstalk level in dB shown in the dashboard is 10 log10(mean leaked power / received power); the default is about -31 dB.

6. Background optical noise

A broadband optical background, specified per nanometre per polarisation (default -45 dBm/nm, switched OFF by default). It is converted to a power spectral density S_b by dividing by the width of 1 nm expressed in Hz, $d\nu = c \times 1\text{ nm} / \lambda^2$, and then treated like extra ASE.

Timing jitter (not a current noise)

Jitter is a random error in the sampling instant (default 2 ps RMS, OFF by default). It is handled in performance.py by sampling the waveform at slightly shifted positions (8 repetitions with random shifts) so that the spread of levels, including the steep edges, increases the level variance.

8.3 The function level_variances

For one logic level with optical power p_level_w and the receiver parameter dictionary p, the function:

1. Chooses ASE and background PSDs only if their flags are on.

2. Computes the signal current I_sig = R P, the mean crosstalk current and the mean ASE current i_ase = R × 2 (S_a + S_b) × Bo.
3. Computes I_tot = I_sig + I_xt + I_ase, which is used for shot noise.
4. Fills a dictionary of variances, one per source, with zero for every disabled source.
5. Returns the dictionary. The caller sums its values.

It is called once for the 1 level and once for the 0 level, giving two total variances var1 and var0.

8.4 Time-domain noise: noise_waveforms

For the signal plots and the Monte-Carlo run, noise is generated as actual waveforms. White Gaussian noise with variance (one-sided PSD) × fs/2 per sample is generated with the NumPy random generator and passed through the same receiver filter. After filtering, its variance equals PSD × noise-equivalent bandwidth, consistent with the formulas above. Signal-dependent noise such as shot and RIN uses a time-varying variance following the instantaneous signal. The function returns each component separately, the combined sum, and the DC offset. This is what the Signals tab plots as separate thermal, shot, RIN, ASE and crosstalk waveforms.

Try it: Turn on the amplifier in the Amplifier tab and open the Noise tab; you will see the ASE bar appear. Then switch off thermal noise and shot noise one at a time and use the Separate and combined effect table to see which source matters most.

Chapter 9: Link Budget and Per-Channel Performance (performance.py)

9.1 Purpose

This is the heart of the simulator. It first tracks the power of every channel through every optical component (the link budget) and then, for each channel, evaluates the receiver: signal levels, noise, SNR, Q-factor, BER and eye opening. It can also run a time-domain Monte-Carlo experiment that actually counts errors.

9.2 Link budget: compute_link

Theory

A link budget is a bookkeeping of power in decibels: received power equals launch power minus all losses plus all gains.

P_rx (dBm) = P_launch (dBm) - MUX loss - fibre loss - connector loss - splice loss - margin - DCF loss + amplifier gain - DEMUX loss

Step-by-step operations in the code

1. **Channel wavelengths and frequencies** are obtained from the WDM module; $\nu = c / \lambda$ for each.
2. **Failed channels** are marked from cfg.fault.failed_channels and their launch power is set to zero (no light).
3. **Launch powers** are converted from dBm to watts.
4. **Fibre loss per channel** = $\alpha(\lambda) \times L$, with the wavelength-dependent attenuation. The default is 0.190 dB/km × 20 km = 3.80 dB.
5. **Connector loss** = number of connectors × loss per connector = 2 × 0.5 = 1.0 dB (the sum of the two ends).
6. **Splice loss** = floor(L / splice spacing) × splice loss = floor(20 / 2) × 0.05 = 0.50 dB.
7. **Margin** (extra safety loss for ageing, repairs, temperature; default 0 dB).
8. **DCF loss** is included only if dispersion compensation is on and the method is DCF (default 5 dB when used).
9. **Number of spans**: if the amplifier is on and in-line, spans = n_inline + 1; otherwise 1.
10. **A list of stages** is built in order: MUX, optional booster, one or more fibre spans (with an in-line EDFA between spans), optional DCF module, optional pre-amplifier, DEMUX.
11. **Power propagation loop.** Starting from the launch power P and a zero ASE density S, each stage is applied. For a loss stage: P ← P × t and S ← S × t, with t = 10^(-loss/10). For

an amplifier: compute the effective gain (with saturation, from the total power of all channels), update $S \leftarrow S \times g + S_{sp}(g)$, and $P \leftarrow P \times g$. The record of each amplifier (gain, n_sp, saturated or not, position) is stored.

12. **Demux data**: the 3 dB bandwidths, the optical noise bandwidths and the crosstalk matrix T are computed from the wdm_system functions and stored.
13. **Everything is returned** in a LinkResult object: p_rx_w, s_sp_rx (ASE PSD at the receiver), total passive loss, power profile along the link for the plot, and so on.

For the default settings the passive loss is 3 (MUX) + 3.80 + 1.0 + 0.5 + 3 (DEMUX) = 11.30 dB, so a 0 dBm launch becomes -11.30 dBm at the receiver. This matches the dashboard value (p_rx_dbm = -11.303 dBm for channel 1). A test (test_received_power_equals_tx_minus_losses) verifies this kind of equality. The small difference between channels (-11.3028 to -11.3114 dBm) arises only because attenuation differs very slightly with wavelength.

9.3 Receiver noise parameters: _rx_noise_params

For channel i this function packs everything the noise equations need:

- **Electrical noise bandwidth** $B_e = B \times \text{rx_bw_frac} \times \text{nebw_factor}$. With B = 10 Gb/s, 0.7 and 1.0645 it is 7.45 GHz.
- **Optical noise bandwidth** $B_o = \max(\text{demux noise bandwidth}, 2\,B_e)$ so that the approximations in the beat-noise formulas remain valid.
- **Background PSD** from the dBm/nm setting.
- **Crosstalk powers** $p_x[j] = P_{rx,j} \times T[i, j]$ for every other healthy channel j; the mean leaked power is their sum and the crosstalk variance is $R^2 \Sigma (p_x \kappa)^2$, with $\kappa = (p1 - p0)/2$ computed for unit average power.
- Receiver constants: R, T, R_L, F, dark current, RIN in linear units, and the ASE density S_a at this channel.

9.4 Noiseless levels: _levels

This helper computes the received current levels without random noise, including the inter-symbol interference caused by dispersion and the receiver filter:

1. Broaden the transmitted waveform: apply a Gaussian filter with width equal to the extra dispersion width (extra_ps converted to samples). This simulates fibre dispersion in the Gaussian-pulse approximation.
2. Convert to current: $I = R \times \text{gain} \times P$, where gain = P_{rx} / P_{launch} for that channel.
3. Apply the receiver filter to get the clean photocurrent waveform i_{cl}.

4. If jitter is on, sample at randomly shifted positions (8 repetitions), otherwise sample exactly at bit centres.
5. Split the samples by the transmitted bit: those where the bit was 1 (s1) and where it was 0 (s0).
6. The mean of s1 is the level I1, the mean of s0 is I0. The standard deviations isi1 and isi0 measure the spread caused by ISI (the pattern dependence).
7. Eye opening: (min of s1 - max of s0) / (I1 - I0). It equals 100% for a perfectly open eye and falls to 0 when closed.

9.5 Evaluating a channel: evaluate_channel

This function is called for every channel. Follow its operations:

1. **Failed check.** If the channel has no light, return the dead-channel metrics (BER 0.5, SNR -99 dB, status FAILED, received power -120 dBm).
2. **Bits and transmitter.** Generate the bit pattern, compute P1 and P0 from the launch power, build the transmit waveform.
3. **Dispersion budget** for this wavelength and bit rate gives extra_ps and the penalty.
4. **Noiseless levels** from _levels with dispersion, and a reference set with zero dispersion (lv_ref).
5. **Noise variances** from level_variances at the effective 1 and 0 optical powers (current divided by R).
6. **Add ISI to the noise variance:** var1 = Σ(noise variances at 1) + isi1², var0 similarly. ISI acts as extra noise because it makes the level uncertain.
7. **Threshold, BER, Q** using the equations of Chapter 6: s1 = √var1, s0 = √var0; threshold from the chosen method; BER by ber_from_levels; Q = (I1 - I0)/(s1 + s0).
8. **SNR in dB:**

$$\textbf{SNR (dB)} = 10 \log_{10} \left(\left((I_1 - I_0)/2 \right)^2 / \left((\text{var}_1 + \text{var}_0)/2 \right) \right)$$

For equal noise this equals Q^2, a relationship verified by test_snr_equals_q_squared_for_equal_noise. In the default case Q = 25.76 and SNR = 28.2 dB.

- **Simulated dispersion penalty:** 10 log10((I1 - I0)_reference / (I1 - I0)_now), the loss of eye amplitude caused by dispersion, to compare with the formula penalty.
- **Crosstalk level in dB** and the metrics dictionary are assembled; the status is classified (Section 9.7).
- **Optional Monte-Carlo (with_waveforms=True):** described next.

9.6 The Monte-Carlo time-domain simulation

Only the channel being inspected is simulated this way, to keep the dashboard fast. The operations are:

1. Generate a longer bit sequence (default 20 000 bits).
2. Build crosstalk waveforms: for every other healthy channel with a significant leaked power, generate its own random bit stream and build its waveform scaled to the leaked power.
3. Generate noise waveforms with noise_waveforms (thermal, shot, RIN, ASE, crosstalk) filtered by the receiver filter.
4. Add the combined noise to the clean photocurrent: noisy = i_cl + combined.
5. Sample at bit centres (with timing jitter if enabled).
6. Compare each sample with the decision threshold to recover bits.
7. Count errors: sim_errors = number of recovered bits not equal to transmitted bits; sim_ber = errors / bits.
8. If zero errors are found the true BER is not zero. The code reports the 95% upper bound 3 / N (the “rule of three”): for 20 000 bits the bound is 1.5 × 10^-4.
9. Save the waveforms for the Signals tab: transmitted power, received power with and without amplifier, noise components, noisy and clean photocurrent, recovered bits, eye traces.

For the default channel the Monte-Carlo run finds 0 errors in 20 000 bits, consistent with the analytic BER being below 10^-15. The test test_monte_carlo_agrees_with_analytic_when_errors_are_frequent confirms agreement in a stressed condition where errors are numerous.

9.7 Status classification and BER display

Condition	Status
Channel has no light	FAILED
BER greater than 1e-3	FAILED
BER greater than 1e-9, or received power below receiver sensitivity (-24 dBm default)	DEGRADED
Otherwise	OK

The function fmt_ber prints values below 1e-15 as “< 1e-15” and never as exactly zero, because the Gaussian approximation cannot honestly claim zero errors.

9.8 Sweeps and comparisons

For the Performance, Noise, Amplifier and Dispersion tabs the module repeats the evaluation with modified configurations, using variant, which makes a deep copy of the configuration and replaces chosen fields:

- **sweep_length:** 24 fibre lengths from 1 km to max(100, 1.1 L), each with and without the amplifier; records received power, SNR, Q and BER.
- **sweep_spacing:** eight DWDM spacings (12.5 to 200 GHz) or six CWDM spacings, with the grid centred on the actual channels; records the worst Q among all channels and the inspected channel Q and crosstalk.
- **sweep_rate:** bit rates 1, 2.5, 5, 10, 25 and 40 Gb/s; records Q, SNR, BER, penalty and eye opening.
- **compare_amplifier:** amplifier OFF versus ON.
- **compare_dispersion:** dispersion off, on without compensation, on with compensation.
- **compare_noise:** no noise (ISI only), each noise source alone, and all configured sources together.
- **ber_vs_snr_curve:** the theoretical BER = 0.5 erfc(√SNR / √2) over 0 to 24 dB, on which the current operating point is marked.
- **pulse_shapes:** a single isolated 1 bit before the fibre, after the fibre without compensation and after the fibre with compensation, all normalised.

Chapter 10: Dispersion and Its Compensation

10.1 What the simulator shows

The Dispersion tab combines Chapter 4 (the physics of pulse broadening) and Chapter 9 (the effect on the eye and BER). The key message printed by the app is: **WDM itself does not remove dispersion; every wavelength is dispersed independently.** A 10 Gb/s pulse is 100 ps wide; in the default case the extra RMS broadening is about 12.8 ps and the eye opening is 95.8% (a small penalty of 0.21 dB). If the length is increased to 80 km, broadening rises four times to about 51 ps, the eye opening falls to 45% and the dispersion penalty reaches 2.1 dB, because the penalty depends on $(B\sigma)^2$. At 60 km the broadening is 38 ps, the eye opening is 68% and Q is already only 3.5, which shows that this default 10 Gb/s link becomes dispersion-limited long before it becomes loss-limited.

10.2 Compensation methods

- **DSP (digital signal processing):** an electronic equaliser at the receiver reverses the dispersion. The model removes a fraction comp_fraction (default 0.9) of the chromatic broadening and adds no optical loss.
- **DCF (dispersion-compensating fibre):** a special fibre with negative dispersion of the opposite sign cancels the accumulated positive dispersion. The model again removes the same fraction but adds the DCF insertion loss (default 5 dB), which lowers received power. The loss is included in compute_link as a loss stage.

Compensation is idealised: only the chromatic term is reduced, by (1 - comp_fraction); modal dispersion and PMD cannot be compensated this way. With the default 20 km span the chromatic term is already small, so compensation changes the numbers very little; it becomes important from about 60 km upward.

10.3 How the pulse plot is produced

pulse_shapes builds an isolated 1 at the middle of 16 bits using a 40 dB extinction ratio so that the 0 level is practically zero. It then applies the Gaussian broadening filter twice, once with the raw width and once with the compensated width, and normalises both by the input peak. Dispersion lowers and widens the pulse; compensation restores most of the peak. The Dispersion tab plots input, no compensation and compensated pulses on the same axes.

Try it: Set length to 80 km, enable dispersion compensation with DSP, and compare the three rows of the Dispersion comparison table. Then switch to DCF and observe that the received power falls by 5 dB although the broadening is the same.

Chapter 11: Traffic Management and Load Balancing (traffic_management.py)

11.1 Purpose and important limitation

The optical layer gives each channel a capacity (its bit rate). The network layer must decide how much customer traffic (demand) each channel carries. Real demand is uneven: one wavelength may be asked to carry 9.5 Gb/s while another carries 2 Gb/s. The module is explicitly a NETWORK-LEVEL simulation. There is no physical optical switching fabric. Traffic can move from one wavelength to another only between channels in the same compatibility group, which models an electronic grooming node or wavelength converter. Every move adds a re-routing latency.

11.2 Terms

Term	Meaning
Demand	Traffic offered to each channel (Gb/s).
Capacity	Bit rate of the channel; zero for a failed channel.
Priority	1 = highest, up to 5. Low priority is dropped first when there is not enough capacity.
Group	Compatibility group; traffic moves only inside a group.
Threshold	Fraction of capacity at which a channel is called congested (default 0.8, i.e. 8 Gb/s on a 10 Gb/s channel).
Carried traffic	Traffic the channel actually transports.
Goodput	Carried traffic after removing packets damaged by bit errors.
Utilisation	Carried / capacity.

11.3 Packet error rate

A packet of N bits is corrupted if at least one bit is wrong. Assuming independent bit errors:

$$\mathbf{PER = 1 - (1 - BER)^N \approx N \times BER \quad (for\ small\ BER)}$$

with N = packet bytes × 8 (1500 bytes = 12 000 bits). The code uses expm1 and log1p so that very small BER values are computed accurately. It returns 0 for BER ≤ 0 and 1 for BER ≥ 0.5. Goodput is carried × (1 - PER).

11.4 The allocation algorithm: allocate_traffic

The function runs three steps in strict order. Each flow is remembered as [origin channel, amount, priority] so that lost traffic can be attributed to its origin.

Step 1 – Protection switching of failed channels

Only if protection is enabled. For each failed channel (highest priority first), its load is moved either to the dedicated backup channel (allowed up to the full capacity) or, if some remains or no backup is set, to the other channels of the same group up to their congestion threshold. The helper move() always transfers the lowest-priority flows first and distributes load to the target with the most free room first.

Step 2 – Adaptive load balancing

Only if enabled. Channels whose load exceeds threshold × capacity are processed in order of priority and then largest excess. The excess above the threshold is moved to other healthy channels of the same group that still have headroom below their own threshold. With the default demands: channel 1 (9.5 Gb/s) has an excess of 1.5 over 8.0, channel 4 (8.8 Gb/s) has an excess of 0.8. These 2.3 Gb/s are moved to the channels with most room: channel 5 (2.0 to 3.5) and channel 3 (3.0 to 3.8). After balancing, no channel is congested.

Step 3 – Capacity enforcement

If a channel still has more load than capacity, the shortage is dropped: with priority scheduling the lowest-priority flows are dropped first; without it the loss is shared proportionally among all flows. Traffic left on a failed channel (capacity zero) is also dropped here.

11.5 Metrics produced

- **Table per channel:** demand, capacity, placed, carried, goodput, utilisation, offered utilisation, dropped, latency and state (OK, CONGESTED, OVERLOAD, FAILED).
- **Latency** with an M/M/1 queue model: service rate μ = capacity / packet size and arrival rate λ = carried / packet size, delay = $1 / (\mu - \lambda)$; the re-route overhead (50 µs) is added in proportion to the fraction of traffic that was moved in. A saturated channel has no finite latency (shown as nan).
- **Totals:** offered, carried, goodput, dropped, loss percentage and the number of congested channels.
- **moved matrix:** moved[i, j] = Gb/s moved from channel i to channel j, shown in the Re-routed traffic table.

Default result: offered 45.3 Gb/s, carried 45.3 Gb/s, dropped 0, nominal capacity 80 Gb/s. This is the distinction between nominal capacity (80 Gb/s, which is just the sum of the bit rates) and received goodput (45.3 Gb/s, because only that much traffic was offered).

The tests check that no channel exceeds capacity, that traffic is conserved (carried + dropped = offered), that balancing reduces congestion and respects groups, that overload drops the lowest priority first, and that a dedicated backup channel is really used.

Try it: In the Traffic tab set the demand of channel 1 to 15 Gb/s. The channel capacity is 10 Gb/s, so at least 5 Gb/s must be moved or dropped. Change the group of channel 2 to 2 and see that traffic can no longer move to it.

Chapter 12: Fault Detection and Recovery (fault_recovery.py)

12.1 Purpose and warning

A channel can fail because of a laser failure, a broken connector or a cut fibre. The module simulates how a network management system notices the failure and restores service. The module warns clearly that all times are SIMULATED from the configured parameters; they are not measurements of real hardware.

12.2 Detection model

A power monitor checks every channel at a fixed polling interval (default 10 ms). To avoid false alarms, a failure is declared only after several consecutive failed polls (the debounce count, default 3). If the failure occurs at time t_fail, the monitor first sees it at the next poll:

$$t_detect = (floor(t_fail / poll) + debounce) \times poll$$

With t_fail = 100 ms, poll = 10 ms and debounce = 3: t_detect = (10 + 3) × 10 = 130 ms. The detection delay is 30 ms. The test test_detection_and_recovery_time_formula verifies this.

12.3 Recovery model

$$t_recover = t_detect + switch\ time$$

With the default 50 ms switching time the traffic is restored at 180 ms, a simulated recovery time of 80 ms after the failure. Recovery occurs only if protection switching is enabled. Whether the traffic is fully recovered depends on whether the remaining channels have enough room: recovered = protection ON and no steady loss remains.

12.4 Three network states

simulate_fault calls allocate_traffic three times with the same demand:

State	Failed channels	Protection	Meaning
Before	None	Not applicable	Normal operation reference
During	Failed ones	OFF	The immediate effect of the failure: the failed traffic is lost
After	Failed ones	As configured	After protection switching: traffic re-routed to the backup channel or the remaining channels

12.5 Metrics

- **rerouted_gbps:** the total moved out of the failed channels in the after state.

- **steady_lost_gbps:** failed demand minus rerouted traffic; whatever cannot be accommodated even after switching.
- **outage_lost_mbit:** failed traffic multiplied by the outage duration (from the failure to recovery, or to the end of the timeline if there is no recovery). For example, a 9.5 Gb/s channel lost for 80 ms loses 760 Mbit.
- **timeline:** a DataFrame in 1 ms steps: aggregate carried traffic equals the before value until t_fail, the during value until t_recover, then the after value; and the traffic of the failed channel.
- **messages:** human-readable text, such as how long detection took and whether recovery was complete.

12.6 The status table

channel_status_table produces the alert text for each channel: “LOSS OF SIGNAL: channel disabled” for a channel that was switched off; “BER above 1e-3: channel unusable” for FAILED by BER; “Degraded: BER > 1e-9 or Rx power below receiver sensitivity” for DEGRADED; blank for OK.

12.7 Optical consequences of a failure

A failed channel carries no light. In the optical model (compute_link) its launch power is zero, so it does not contribute crosstalk to its neighbours (their Q increases slightly) and it reduces the total power entering a saturated amplifier (the remaining channels may get more gain). The dashboard caption says exactly this.

Try it: In the Failure tab pick channel 4 and press Trigger failure. Compare recovery with Reroute to remaining channels and with Dedicated backup channel 8. Set the poll period to 50 ms and see the detection time and the lost Mbit rise.

Chapter 13: The Orchestrator (simulation.py)

13.1 Purpose

simulation.py does no new physics. It calls the other modules in the correct order and gathers everything into one SimResults object that the dashboard displays.

13.2 The function run_simulation step by step

1. **compute_link(cfg)** – the link budget with the configured amplifier, and again with the amplifier forced off (link_noamp) for the before / after comparison.
2. **noise_flags(cfg)** – the on / off switches of the noise sources.
3. **evaluate_all** – the analytic metrics of every channel, producing the results table.
4. **Choose the inspected channel.** The channel chosen in the sidebar is used. If it has failed (and not all have), the first healthy channel is used instead and a note is stored. If all channels have failed, only the dead metrics are produced and the dashboard shows an error and a Restore button.
5. **Detailed evaluation** of the chosen channel with the Monte-Carlo and waveform generation.
6. **collect_issues** – gathers all warnings and messages (fibre, WDM, amplifier below the quantum limit, saturated amplifiers, receiver bandwidth too high for the number of samples per bit, ASE enabled while the amplifier is off, responsivity unrealistic for POF / PCS at short wavelengths).
7. **Traffic** – allocate_traffic twice, once without balancing and once with balancing, to show the benefit; BER values from the table are passed to compute packet errors.
8. **simulate_fault** – the fault timeline and recovery.
9. **channel_status_table** – status and alert text.
10. **pipeline_status** – a coloured state for each of the eleven pipeline blocks for the overview diagram (OK, warn, error, off).
11. **Sweeps and comparisons** (if requested): length, spacing and rate sweeps; amplifier, dispersion and noise comparisons; pulse shapes.
12. **Return SimResults** including nominal capacity (sum of bit rates) and received goodput (from the balanced traffic result).

The dashboard wraps run_simulation in a Streamlit cache keyed by the JSON string of the configuration, so repeated visits to a tab or unchanged settings cost nothing.

Chapter 14: The Dashboard, Tab by Tab (app.py)

14.1 Purpose

app.py is the user interface built with Streamlit. Streamlit re-runs the whole script from top to bottom every time the user touches a widget. The file is therefore organised as: state handling, helper functions, sidebar, simulation call, then twelve tabs.

14.2 State handling

- **st.session_state** stores every widget value under a key of the form section.field, for example fiber.length_km or amp.gain_db. init_state creates the keys from the defaults of SystemConfig only if they do not exist yet, so user values survive each re-run.
- **Tables.** The per-channel tables for the WDM tab (bit rate, launch power, custom wavelength) and the Traffic tab (demand, priority, group) are stored as pandas DataFrames in session state and edited with st.data_editor. edit_table writes the edited rows back into the full 16-row table so that values of hidden channels are not lost when the number of channels changes.
- **collect_config** reads all widget values and builds a SystemConfig. The list-type fields come from the DataFrames, the failed channels come from session state, and the optional manual attenuation / dispersion override is set to None unless the manual checkbox is ticked. If the operating wavelength is set to follow the WDM channels, it becomes the average of the channel wavelengths.
- **reset_defaults** deletes all section keys and calls init_state(force=True).
- **Callbacks.** on_fiber_choice loads the preset defaults when material, structure or profile changes and clears the manual overrides; sync_len_slider and sync_len_number keep the length slider and the numeric box equal; on_wdm_edit automatically turns on Custom wavelengths when the user types in that column; fault_trigger and fault_restore edit the failed channel list.

14.3 The sidebar and the run logic

1. The sidebar has Run Simulation, Auto-update, Reset to Defaults, Channel inspected in detail, Reproducible random seed and the Seed value.
2. cfg_now is the configuration from the current widgets.
3. If Run was pressed, or Auto-update is on, or nothing has been applied yet, the current configuration JSON is saved as applied_cfg. Otherwise the old applied configuration is kept, and the sidebar warns that settings changed and the Run button must be pressed.
4. Blocking errors (from validate_fiber and validate_wdm) stop the page with a red message.

5. cached_run(cfg.to_json()) returns the SimResults; Streamlit caches up to 8 results (max_entries = 8).
6. If all channels have failed, an error and a Restore button are shown and the page stops.

14.4 Tab 1 – Overview

Shows the block diagram of the whole system with colour-coded stages, five key metrics (nominal capacity, received goodput, number of OK / degraded / failed channels, worst-channel Q and received power of the inspected channel), the list of warnings and the channel status table with alerts. The caption explains the difference between nominal capacity and goodput.

14.5 Tab 2 – Fibre

Selection of material, structure and index profile; length slider (1 to 100 km) with a numeric box (up to 400 km); core and cladding diameter; indices; operating wavelength (automatic or manual); PMD coefficient; modal bandwidth; manual attenuation and dispersion; connector and splice losses and margin. A table lists the calculated numerical aperture, V-number, number of modes, cutoff wavelength, attenuation, dispersion D, modal dispersion, PMD and fibre loss. Two plots show attenuation versus wavelength (logarithmic axis) and chromatic dispersion versus wavelength across the valid range of the material.

14.6 Tab 3 – WDM

Scheme (DWDM or CWDM), number of channels (1 to 16), custom wavelength switch, grid parameters (centre and spacing for DWDM; start and spacing for CWDM), MUX and DEMUX losses, demux bandwidth fraction, filter order, isolation floor and the editable per-channel table. It displays the grid with wavelengths in nm and frequencies in THz, the validation messages and the total nominal data rate.

14.7 Tab 4 – Tx/Rx

Modulation, extinction ratio, rise time, samples per bit, displayed bits, Monte-Carlo bits, threshold type, laser RIN, linewidth, spectral width, responsivity, receiver bandwidth ratio, receiver filter (with the Butterworth order when selected), sensitivity, temperature, load resistance, receiver noise figure and dark current. It displays the computed receiver 3 dB bandwidth in GHz and the noise-equivalent bandwidth factor.

14.8 Tab 5 – Noise

Seven switches (thermal, shot, RIN, ASE, crosstalk, background, jitter), conditional inputs for background level and jitter value, a bar chart of the RMS contribution of each noise source at logic

1 for the inspected channel, and the comparison table (no noise, each source alone, all sources) giving Q, SNR and BER.

14.9 Tab 6 – Amplifier

Switch, placement, gain, noise figure, number of in-line amplifiers, saturation switch and power limit. It shows the OFF versus ON comparison (received power, SNR, Q, BER and ASE noise), the power-profile plot along the link, received-signal waveforms before and after amplification, the amplifier input and output waveforms and a line per amplifier giving effective gain, n_sp and whether it is output-power limited. An information box explains that amplification helps when thermal noise dominates but adds ASE.

14.10 Tab 7 – Dispersion

Dispersion on / off, compensation on / off, method (DSP or DCF), compensated fraction and DCF loss. Five metrics: D, chromatic RMS, modal / PMD RMS, net broadening after compensation, dispersion penalty. A pulse plot and the three-row comparison table. An information box states that WDM does not remove dispersion.

14.11 Tab 8 – Signals

This is the visual laboratory. For the inspected channel, in a common bit window and at the same bit rate, it plots in order: the data bits; the transmitted optical power; the received optical power before and after the amplifier; amplifier input and output; separate waveforms of thermal, shot, laser RIN, ASE and crosstalk noise; the combined noise; clean versus noisy photocurrent; the recovered bits against the transmitted bits; the noisy eye diagram with Q and eye opening printed on it; the optical spectrum of all channels; and the received power of all channels in bars. The final line reports the Monte-Carlo errors and the analytic BER side by side, with the rule-of-three bound when no errors are seen.

14.12 Tab 9 – Performance

Eleven metrics (Tx power, Rx power, total passive loss, SNR, BER, crosstalk, Q, eye opening, pulse broadening, dispersion penalty, aggregate goodput), the per-channel table with a coloured status icon, and the plots: BER versus SNR with the operating point; BER, received power and SNR versus fibre length with and without the amplifier; Q versus channel spacing; Q and eye versus data rate; bar comparisons for amplifier, dispersion and noise.

14.13 Tab 10 – Traffic

Switches for adaptive balancing and priority scheduling, congestion threshold, re-route latency, and the editable demand / priority / group table. Five metrics (offered, carried, goodput, dropped,

congested channels) with differences relative to the no-balancing case, bar charts before and after balancing, the detailed table with coloured states and the list of re-routed traffic (from channel, to channel, Gb/s).

14.14 Tab 11 – Failure

Choose a channel and press Trigger failure; press Restore to clear all failures; switch protection and choose the backup mode and channel; enter failure time, poll period, debounce, switch time and timeline end. Messages, five metrics (detected, recovered, simulated recovery time, lost Mbit, still lost Gb/s), the status table, the timeline plot and the before / during / after traffic charts.

14.15 Tab 12 – Summary

A text summary of the configuration and the result; three downloads: channel results as CSV, configuration as JSON and a report in Markdown; the parameter provenance table; and a paragraph of limitations. The report includes the configuration JSON and a Markdown table of results if the tabulate package is available.

14.16 Helper details worth knowing

- **plot()** gives each Plotly chart a unique key because the same figure may appear in two tabs and Streamlit would otherwise raise a duplicate-widget error.
- **pretty()** renames code-style column names (for example p_rx_dbm) to readable names with units (Received power (dBm)).
- **issues_box()** shows error, warning or info messages in red, yellow or blue boxes.
- A small CSS block enlarges fonts by 12.5% and styles the title and metrics.

Chapter 15: The Plotting Layer (visualizations.py)

This file only draws. Every function takes data from the results and returns a Plotly figure; none performs physics. The common helper _base sets the title, axis labels, height and style. _disabled returns an empty figure with an explanatory text when a feature is turned off (for example, ASE noise while the amplifier is off), so that the layout never jumps.

Function	What it draws
fig_block_diagram	The overview block diagram with coloured stage status, link loss annotations and the channel summary.
fig_pipeline	A simple row of stage boxes with status colours.
fig_data	The data bit stream as a stepped line.
fig_optical	Transmitted optical power in time.
fig_rx_before_after	Received signal without and with the amplifier.
fig_amp_in_out	Waveform at the amplifier input and output.
fig_noise	One noise component in time (thermal, shot, RIN, ASE, crosstalk) or a disabled notice.
fig_combined_noise	The sum of all enabled noise currents.
fig_clean_vs_noisy	Photocurrent with and without noise, with the decision threshold.
fig_recovered	Recovered bits compared with transmitted bits, marking errors.
fig_eye	Eye diagram from overlaid traces, optionally with metrics text.
fig_spectrum	Optical spectrum of the channels with the ASE floor.
fig_rx_power_bars	Received power of every channel.
fig_power_profile	Power in dBm against distance along the link, showing amplifier jumps.
fig_ber_vs_snr	Theoretical BER curve and the operating point.
fig_lines, fig_bars	Generic line and bar plots used by the sweeps and comparisons.
fig_noise_contrib	Bar chart of the RMS noise by source.
fig_pulses	Input, dispersed and compensated pulse shapes.
fig_traffic, fig_throughput	Per-channel load before and after balancing; throughput comparison.
fig_fault_timeline	Aggregate and failed-channel traffic in time with failure, detection and recovery markers.

Chapter 16: Examples and Automated Tests

16.1 run_examples.py

Running python run_examples.py writes a folder examples with six JSON configurations and a sample_results.md file. The six configurations are:

Name	System
dwdm_smf_20km	The default: 8-channel DWDM over 20 km of standard single-mode fibre.
cwdm_smf_40km	8-channel CWDM starting at 1471 nm, 20 nm spacing, 40 km, demux bandwidth fraction 0.6.
dwdm_smf_150km_amplified_dsp	Long-haul: 150 km, three in-line EDFAs of 20 dB gain and 5 dB noise figure, 20 dBm saturation, DSP compensation of 95%.
mmf_graded_swdm4_50m	Short-wave WDM, 4 channels (850, 880, 910, 940 nm), 50 m of graded-index multimode fibre, responsivity 0.5 A/W.
pof_si_2ch_50m	Plastic optical fibre, two visible channels (520 and 650 nm) at 50 Mb/s, 50 m.
pcs_si_2ch_100m	Plastic-clad silica, two channels (850 and 1300 nm) at 50 Mb/s, 100 m.

It also creates two tables: the effect of fibre material and length (silica single-mode, silica multimode graded and step, PCS and POF at lengths from 50 m to 10 km) and the effect of length, amplifier and DSP compensation for the DWDM system at 20, 60, 80, 100 and 150 km. These tables show, for example, why step-index multimode fibre is useful only for tens of metres, and why plastic fibre is limited by its very high attenuation.

16.2 The automated tests

The tests act as the quality check of the simulator. Each one states a physical expectation and verifies that the code meets it.

Test file	What it proves
test_fiber.py	Silica attenuation matches the textbook windows; attenuation depends on wavelength and the override scales it; G.652 dispersion values; NA and V-number; received power equals transmitted power minus the losses; loss rises with length; dispersion grows with length and compensation helps; step-index modal dispersion is larger than graded and single-mode has none; changing the fibre configuration changes parameters; invalid combinations produce warnings; mode label checked against V; wavelength outside the material range produces a warning.

Test file	What it proves
test_wdm.py	Frequency and wavelength conversion round trip; 100 GHz DWDM spacing is about 0.8 nm; CWDM grid is 20 nm; duplicates and bad spacing are rejected; total nominal rate is the sum of the channel rates; the demux crosstalk matrix has the expected properties.
test_noise_amp.py	Thermal and shot formulas; a disabled noise source contributes exactly zero; shot noise grows with power and RIN with its square; the amplifier n_sp and ASE PSD formulas; gain applied in the link and saturation limits the output; ASE appears only with an amplifier; toggling noise changes the result.
test_ber_dispersion.py	BER reference values for Q; BER curve monotonic and equal to the Q^2 relation; SNR equals Q^2 for equal noise; zero errors report an upper bound; Monte-Carlo agrees with the analytic BER when errors are frequent; BER worsens with length and the amplifier changes power; dispersion closes the eye and compensation reopens it; crosstalk increases when spacing shrinks; a failed channel reduces the crosstalk of its neighbours; the spacing sweep is centred on custom channels.
test_traffic_fault.py	No channel exceeds capacity and traffic is conserved; balancing reduces congestion and respects threshold and groups; overload drops the lowest priority first; protection recovers traffic and no protection loses it; dedicated backup is used; detection and recovery time formulas; failed-channel status and dead metrics; the inspected failed channel falls back to a healthy one.
test_app.py	Using Streamlit AppTest (no browser): the dashboard runs, reconfigures for every material / structure / profile, switches to CWDM, changes channel count and length without exceptions; the failure and restore buttons work; custom wavelengths flow through every view and the operating wavelength follows their average (1380 nm in the test).

Run all tests with the command pytest in the project folder. Note that test_app.py locates app.py one directory above the test file (os.path.join(..., "..", "app.py")), so the tests are expected to sit in a tests subfolder next to app.py.

Chapter 17: A Complete Worked Example with the Default Settings

This chapter follows channel 1 of the default configuration from launch to decision, using the real numbers produced by the simulator. It is the best way to see how all the previous chapters fit together. All values come from running run_simulation on SystemConfig() with default settings.

17.1 Step 1 – The setup

- DWDM, 8 channels, 100 GHz spacing, channel 1 at 1547.32 nm, 10 Gb/s, launch power 0 dBm.
- Standard single-mode silica fibre, 20 km, no amplifier, dispersion on, no compensation.

17.2 Step 2 – Fibre properties

Quantity	Value	Equation used
Numerical aperture	0.1199	$\text{sqrt}(1.4682^2 - 1.4633^2)$
V-number	1.99	$2\pi\, a\, NA / \lambda$
Cutoff wavelength	1284 nm	$2\pi\, a\, NA / 2.405$
Single-mode?	Yes ($V < 2.405$)	Comparison
Attenuation	0.190 dB/km	$A/\lambda^4 + B + IR + OH$
Chromatic dispersion D	17.14 ps/nm/km	$(S0/4)(\lambda - \lambda0^4/\lambda^3)$

17.3 Step 3 – Link budget

Item	Loss (dB)
MUX	3.00
Fibre: 0.190 × 20	3.80
Connectors: 2 × 0.5	1.00
Splices: floor(20/2) × 0.05	0.50
Margin	0.00
DEMUX	3.00
Total passive loss	11.30
Launch → received	0 dBm → -11.30 dBm

The received power -11.30 dBm is 13 dB above the receiver sensitivity of -24 dBm, so the received power is not a problem.

17.4 Step 4 – Dispersion

- Input RMS width of the bit: 100 ps / √12 = 28.87 ps.
- Source spectral width: σ_λ = 0.0372 nm.
- Chromatic broadening: 17.14 × 20 × 0.0372 = 12.75 ps. PMD = 0.45 ps. Modal = 0.
- Total extra width = √(12.75² + 0.45²) = 12.76 ps; output width = 31.56 ps.
- Dispersion penalty = 5 log10(1 + 2π (10^10 × 12.76 × 10^-12)²) = 0.21 dB.
- Eye opening (ISI) = 95.8%.

17.5 Step 5 – Receiver levels and noise

Quantity	Value
Current at logic 1 (I1)	118.45 µA
Current at logic 0 (I0)	14.81 µA
Thermal noise RMS	0.70 µA
Shot noise RMS at logic 1	0.53 µA
Laser RIN noise RMS at logic 1	0.32 µA
Crosstalk noise RMS at logic 1	0.015 µA
ASE noise	0 (amplifier off)
Total σ1 (noise + ISI)	2.03 µA
Total σ0 (noise + ISI)	1.995 µA

A useful consistency check: the 1 and 0 currents correspond to P1 = 118.45/(0.9 × gain) and the gain here is 0.0741 (= -11.3 dB). At the launch, P1 = 1.82 mW. Then I1 ≈ 0.9 × 0.0741 × 1.82 mW = 121 µA before the receiver filter and dispersion reduce the peak slightly to 118.45 µA, and I0 ≈ 0.9 × 0.0741 × 0.182 mW = 12.1 µA, raised to 14.81 µA by pattern-dependent ISI. These numbers are consistent.

17.6 Step 6 – Q, SNR and BER

Q = (118.45 - 14.81) / (2.03 + 1.995) = 25.76

BER = 0.5 erfc(25.76 / √2) ≈ 1.3 × 10^-146 (reported as < 1e-15)

SNR = 10 log10(51.82² / 4.05) = 28.2 dB

Since the BER is far below 10^-9, the channel status is OK. The Monte-Carlo run sends 20 000 bits and finds 0 errors, giving a statistical upper bound of 1.5 × 10^-4; the analytic value is a much

sharper (but model-dependent) estimate. This is why the application shows both values side by side.

17.7 Step 7 – What the noise comparison tells us

Thermal noise is the largest contributor at 20 km, so this short link is thermal-noise limited and has a huge margin (Q = 25.8 against the target of 6). But the simulator shows an important lesson when the length grows. Running the same default system at other lengths gives the following results for channel 1:

Length (km)	Amplifier	Rx power (dBm)	Q	BER	Broadening (ps)	Eye (%)
20	None	-11.3	25.76	< 1e-15	12.8	96
20	15 dB pre-amp	3.7	27.38	< 1e-15	12.8	96
60	None	-19.9	3.49	2.45e-04	38.3	68
60	15 dB pre-amp	-4.9	3.91	4.68e-05	38.3	68
80	None	-24.2	1.65	4.99e-02	51.0	45
80	15 dB pre-amp	-9.2	2.29	1.11e-02	51.0	45
100	None	-28.5	0.67	2.53e-01	63.8	17
150	None	-39.3	0.04	4.83e-01	95.7	0

Two conclusions follow. First, Q falls below 6 somewhere between 20 and 60 km, long before the received power reaches the -24 dBm sensitivity, because chromatic dispersion closes the eye (68% at 60 km, 45% at 80 km). Second, the 15 dB pre-amplifier raises received power by 15 dB but improves Q only slightly at 60 and 80 km (3.49 to 3.91 and 1.65 to 2.29), since the link is dispersion-limited, not power-limited. In this model such a link needs dispersion compensation (Chapter 10) much more than extra power. These values were obtained by running the simulator with default settings except for length and amplifier; they are simulated numbers, not measurements.

17.8 Step 8 – Traffic and network layer

Channel	Demand (Gb/s)	Carried after balancing (Gb/s)	Action
1	9.5	8.0	1.5 moved out (above 80% threshold)
2	6.0	6.0	None
3	3.0	3.8	0.8 moved in

Channel	Demand (Gb/s)	Carried after balancing (Gb/s)	Action
4	8.8	8.0	0.8 moved out
5	2.0	3.5	1.5 moved in
6	5.0	5.0	None
7	4.0	4.0	None
8	7.0	7.0	None

Offered 45.3 Gb/s, carried 45.3 Gb/s, dropped 0. Nominal capacity 80 Gb/s, received goodput 45.3 Gb/s (the BER is so low that packet errors are negligible). Without balancing, channels 1 and 4 would be flagged CONGESTED; with balancing there are no congested channels.

17.9 Step 9 – A fault scenario

Trigger failure of channel 1 at t = 100 ms with protection on. Detection time = (floor(100/10) + 3) × 10 = 130 ms. Recovery at 130 + 50 = 180 ms, a simulated recovery time of 80 ms. Lost during the outage: 9.5 Gb/s × 0.080 s = 0.76 Gb = 760 Mbit. After switching, the 9.5 Gb/s is re-routed to the remaining channels of the group up to their 80% threshold; if there is enough headroom the traffic is fully recovered.

Chapter 18: Limitations, Provenance and Practice Exercises

18.1 Limitations of the model

Understanding what the simulator does not model is as important as understanding what it does. The application states these limits itself in its Summary tab:

- Only intensity-modulation with direct detection (OOK) is modelled; there are no coherent formats such as QPSK or QAM.
- Fibre nonlinearities (self-phase modulation, cross-phase modulation, four-wave mixing, stimulated Raman scattering) are not modelled. These limit high-power long-haul DWDM in real systems.
- The EDFA gain is flat across wavelength and instantaneous; real EDFAs have a gain tilt and dynamics.
- Dispersion is a Gaussian-pulse RMS approximation, with widths added in quadrature; the compensation removes a fixed fraction.
- BER is based on a Gaussian-noise approximation. A zero-error Monte-Carlo run gives only an upper bound.
- The demux filter does not cause ISI in the model; its bandwidth is checked only by a warning.
- Traffic balancing and protection switching are network-level simulations; there is no physical switch fabric, and the recovery time is derived from parameters, not measured.
- No value is a measurement of real hardware.

18.2 Parameter provenance in brief

Type	Examples
Sourced	CODATA constants; the form of the silica attenuation law; G.652 dispersion law with S0 = 0.092 and λ0 = 1313 nm; EDFA noise theory; the noise formulas; BER = 0.5 erfc(Q/√2); dispersion penalty formula
Calculated	Step-index modal dispersion L n1 Δ / (2√3 c); V-number, NA, cutoff; all simulation outputs
Assumed	POF attenuation table; PCS attenuation; fibre presets; receiver values (responsivity, load, noise figure, dark current); RIN, linewidth, jitter; EDFA gain, noise figure and saturation; connector, splice and margin; MUX / DEMUX loss, bandwidth and isolation; protection switching and monitoring times

18.3 Practice exercises

Do these in the dashboard. Each one reinforces one chapter of this tutorial.

1. **Attenuation window (Chapter 4).** On the Fibre tab look at the attenuation plot for silica. Identify the OH peak near 1383 nm and the minimum near 1550 nm. Explain why the default WDM channels are around 1550 nm.
2. **Zero dispersion (Chapter 4).** Make a CWDM system around 1310 nm and compare the broadening with the DWDM system at 1550 nm at 40 km. Which is dispersion-limited, and why?
3. **Spacing and crosstalk (Chapter 5).** Set DWDM spacing to 200, 100, 50, 25 and 12.5 GHz. At which spacing does the crosstalk level rise above -20 dB? Compare with the Performance tab sweep.
4. **Extinction ratio (Chapter 6).** Reduce the extinction ratio from 10 dB to 3 dB. Calculate the new P1 and P0 by hand using the formula of Section 6.3 and check I1 - I0 in the channel metrics.
5. **Amplifier trade-off (Chapter 7).** At 100 km, compare no amplifier, a pre-amplifier of 15 dB and an in-line amplifier set (n = 3). Which placement gives the best Q? Why?
6. **Noise ranking (Chapter 8).** Using the Noise tab, rank the sources by Q improvement when each is removed at 20 km and at 80 km with the amplifier on.
7. **Link budget by hand (Chapter 9).** For 50 km without amplifier compute the received power by hand (attenuation 0.19 dB/km, 2 connectors at 0.5 dB, splice every 2 km at 0.05 dB, MUX and DEMUX 3 dB each). Check against the dashboard. Is it above -24 dBm?
8. **Dispersion compensation (Chapter 10).** At 100 km compare DSP and DCF. Why does DCF make the received power lower?
9. **Load balancing (Chapter 11).** Set all demands to 9 Gb/s. How many channels are congested with and without balancing? What happens when the demand is raised to 11 Gb/s on one channel?
10. **Protection (Chapter 12).** Fail three channels at once with demands at 9 Gb/s each. Can the remaining channels absorb the traffic? Note steady lost Gb/s.
11. **Different fibres (Chapter 16).** Load the POF example values (650 nm, 50 m). Increase the length until the Q falls below 6. Compare with silica at the same bit rate.

Appendix: Symbols, Formulas and Glossary

A.1 Table of symbols

Symbol	Meaning	Unit
c	Speed of light	m/s
λ, ν	Wavelength, optical frequency	nm, Hz
n_core, n_clad	Refractive indices of core and cladding	-
NA	Numerical aperture	-
V	Normalised frequency (V-number)	-
Δ	Relative index difference	-
α	Attenuation coefficient	dB/km
D	Chromatic dispersion parameter	ps/(nm km)
σ	RMS width (pulse) or RMS noise current (receiver), depending on context	ps or A
R	Photodiode responsivity	A/W
P, I	Optical power, photocurrent	W, A
P1, P0, I1, I0	Power and current at logic 1 and logic 0	W, A
r	Extinction ratio P1/P0	-
G, NF, n_sp	Amplifier gain, noise figure, spontaneous emission factor	-, dB, -
S_sp	ASE power spectral density per polarisation	W/Hz
Be, Bo	Electrical and optical noise-equivalent bandwidths	Hz
k, T, R_L, F	Boltzmann constant, temperature, load resistance, receiver noise factor	J/K, K, ohm, -
q	Electron charge	C
Q	Q-factor (I1 - I0)/(σ1 + σ0)	-
T_ij	Demux crosstalk matrix element	-

A.2 Key formulas in one place

Topic	Formula
dB and dBm	dB = 10 log10(P_out/P_in); dBm = 10 log10(P/1 mW)
Wavelength-frequency	$c = \lambda \nu$; $\Delta\lambda = \lambda^2 \Delta\nu / c$
NA, V, cutoff	$NA = \sqrt{(n1^2 - n2^2)}$; $V = 2\pi a NA/\lambda$; $\lambda c = 2\pi a NA / 2.405$
Attenuation	$\alpha(\lambda) = A/\lambda^4 + B + IR + OH$; loss = α L

Topic	Formula
Chromatic dispersion	$D = (S0/4)(\lambda - \lambda0^4/\lambda^3)$; $\sigma_chrom = \|D\| L \sigma_\lambda$
Modal dispersion	Step: $L\ n1\ \Delta / (2\sqrt{3}\ c)$; graded: $L\ n1\ \Delta^2 / (20\sqrt{3}\ c)$
PMD	$PMD_coeff \times \sqrt{L}$
Dispersion penalty	$5 \log10(1 + 2\pi (B\sigma)^2)$
Demux filter	$T = \exp(-\ln2\ (2\|\Delta f\|/BW)^{(2n)})$; $B_o = BW\ \Gamma(1+1/2n)/(\ln2)^{(1/2n)}$
Tx levels	$P0 = 2\ P_avg/(d\ r + 2 - d)$; $P1 = r\ P0$
Thermal noise	$4\ k\ T\ F\ Be / R_L$
Shot noise	$2\ q\ (I + I_dark)\ Be$
RIN noise	$RIN\ I^2\ Be$
EDFA	$n_sp = (NF\ G - 1)/(2(G - 1))$; $S_sp = n_sp\ (G - 1)\ h\ \nu$
Signal-ASE / ASE-ASE	$4R^2P\ S_sp\ Be$; $4R^2\ S_sp^2\ Bo\ Be\ (1 - Be/2Bo)$
Q and BER	$Q = (I1 - I0)/(\sigma1 + \sigma0)$; $BER = 0.5\ erfc(Q/\sqrt{2})$
SNR	$((I1 - I0)/2)^2 / ((\sigma1^2 + \sigma0^2)/2)$
Packet error rate	$1 - (1 - BER)^N$
Detection time	$(floor(t_fail/poll) + debounce) \times poll$

A.3 Glossary

Term	Meaning
ASE	Amplified spontaneous emission: broadband optical noise generated by an optical amplifier.
BER	Bit error rate: the fraction of received bits that are wrong.
CWDM / DWDM	Coarse and dense wavelength division multiplexing.
DCF	Dispersion-compensating fibre: fibre with opposite dispersion sign.
DSP	Digital signal processing; here, an electronic equaliser for dispersion.
EDFA	Erbium-doped fibre amplifier.
Eye diagram	Overlay of many bit periods of the received signal; a wide opening means reliable detection.
Extinction ratio	Ratio of the optical power in a 1 to that in a 0.
Goodput	Useful traffic delivered, after dropping and packet errors.
ISI	Inter-symbol interference: neighbouring bits overlapping in time.
IM/DD	Intensity modulation with direct detection.
MUX / DEMUX	Multiplexer combines channels into one fibre; demultiplexer separates them.

Term	Meaning
NRZ / RZ	Non-return-to-zero and return-to-zero pulse formats.
OOK	On-off keying.
PMD	Polarisation-mode dispersion.
POF / PCS	Plastic optical fibre / plastic-clad silica.
Q-factor	Quality measure linking signal separation and noise to BER.
RIN	Relative intensity noise of the laser.
Responsivity	Photocurrent per optical watt of the photodiode.
Crosstalk	Unwanted leakage of neighbouring channels into a channel.
Protection switching	Automatic re-routing of traffic when a channel fails.